INSTITUTO SUPERIOR DE TRANSPORTES E COMUNICAÇÕES
Licenciatura em Engenharia e Ciências dos Computadores — LECC 31
Sistemas Distribuídos e Paralelos

Teste 2 — Relatório Técnico

SCAD

Sistema de Compras Automáticas Distribuídas

Membros do grupo:
Bill Arsenio Ndayisaba
Bruna Fransciso
Sousa Marufo

Docente: MSc. Sérgio J. Sambo

Repositório: https://github.com/Bill-Ars/scad

Maputo, novembro de 2026

Índice

1. Introdução e objetivos **3**
2. Arquitetura do sistema **3**
2.1 Fluxo de uma compra automática 4
2.2 Utilização do sistema 5
3. Sistemas Operativos Distribuídos **6**
3.1 Exclusão mútua: algoritmo de Ricart-Agrawala 6
3.2 Sincronização de relógios: algoritmo de Berkeley 7
4. Sistemas de Bases de Dados Distribuídos **7**
4.1 Modelo de dados e alocação 7
4.2 Fragmentação horizontal 8
4.3 Replicação mestre-escravo 8
4.4 Consistência: Two-Phase Commit 9
5. Sistema de Ficheiros Distribuído **9**
6. Java / CORBA **10**
7. Serviços Web Distribuídos **11**
8. Testes de tolerância a falhas **11**
9. Limitações e trabalho futuro **12**
10. Conclusão **13**
11. Referências bibliográficas **13**
Anexo A — Instalação e utilização **14**

1. Introdução e objetivos

O presente relatório descreve o desenho e a implementação do **SCAD — Sistema de Compras Automáticas Distribuídas**. O sistema gere o stock de três armazéns regionais (Maputo, Beira e Nampula). Os clientes, pessoas particulares ou instituições como hospitais e bancos, fazem e pagam pedidos a um armazém; o stock desce e, sempre que um produto atinge o stock mínimo, o SCAD emite, de forma automática e descentralizada, uma ordem de compra a um fornecedor externo.

O objetivo principal é aplicar, num sistema funcional, os conceitos fundamentais de sistemas distribuídos exigidos no enunciado: exclusão mútua e sincronização de relógios (SOD — Sistemas Operativos Distribuídos), fragmentação, replicação e consistência transacional (SBDD — Sistemas de Bases de Dados Distribuídos), armazenamento partilhado tolerante a falhas (DFS — Distributed File System), comunicação por objetos distribuídos (CORBA — Common Object Request Broker Architecture) e integração por serviços web (REST — Representational State Transfer, e SOAP — Simple Object Access Protocol).

O sistema foi implementado em Java (Spring Boot 4 sobre Java 21 e JacORB sobre Java 8), PostgreSQL 16 e Apache Hadoop 3, e é executado em 20 contentores Docker orquestrados por Docker Compose. O código e as instruções de instalação encontram-se no repositório indicado na capa.

2. Arquitetura do sistema

Cada nó do sistema é um contentor Docker com processo, sistema de ficheiros e endereço IP próprios, ligado aos restantes apenas pela rede virtual `scad-net`. Esta propriedade permite tratar cada contentor como um nó independente, que comunica exclusivamente por mensagens e que pode falhar isoladamente (`docker compose stop`). A Figura 1 apresenta a arquitetura e a Tabela 1 resume as camadas.

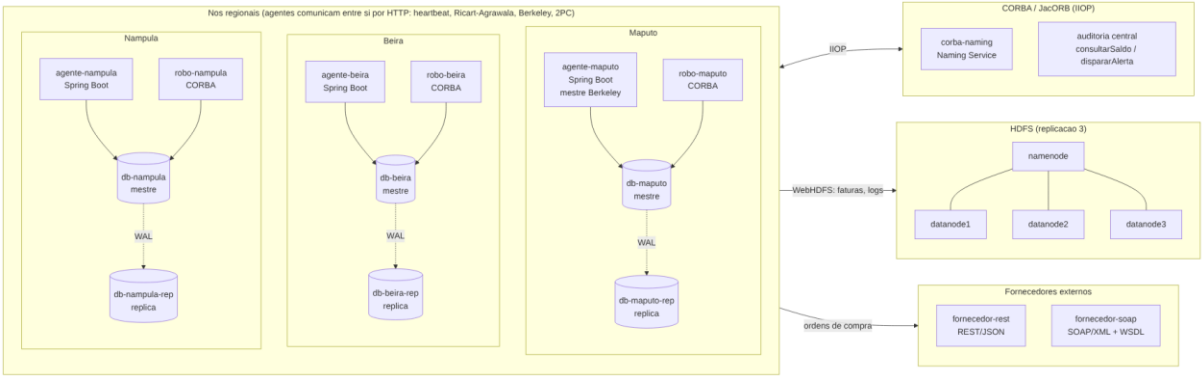


Figura 1 — Arquitetura do SCAD (20 contentores numa rede Docker)

Camada	Serviços	Tecnologia
Agentes regionais e sistema web	`agente-maputo`, `agente-beira`, `agente-nampula`	Spring Boot 4.1, Java 21
Base de dados distribuída	3 mestres (`db-*`) e 3 réplicas (`db-*-rep`)	PostgreSQL 16, replicação por streaming

Camada	Serviços	Tecnologia
Comunicação interna legada	`corba-naming`, `auditoria`, `robo-*`	JacORB 3.9, Java 8, IIOP
Sistema de ficheiros	`namenode`, `datanode1..3`	Apache Hadoop 3 (HDFS), WebHDFS
Fornecedores externos	`fornecedor-rest`, `fornecedor-soap`	Spring Boot, JSON, SOAP 1.1/WSDL

Tabela 1 — Camadas e serviços

O mesmo código de agente é executado três vezes; a região, o desvio de relógio, a base de dados e os pares de cada nó são definidos por variáveis de ambiente. Os nós encontram-se pelo nome (DNS interno do Docker), o que torna a localização transparente. Os fornecedores são serviços independentes: o SCAD só conhece o endereço e o tipo de integração de cada um.

2.1 Fluxo de uma compra automática

A Figura 2 resume a interação entre componentes numa compra automática, que integra as cinco componentes do enunciado:

1. O agente deteta, no seu fragmento local, que um produto atingiu o mínimo (verificação de 10 em 10 s e logo após cada pedido); em paralelo, o robô de stock regional envia um alerta à auditoria central por CORBA.
2. O agente pede a secção crítica do lote `fornecedor:produto` com o algoritmo de Ricart-Agrawala.
3. Dentro da secção crítica, verifica se outro nó já encomendou o lote; em caso afirmativo, não duplica a compra.
4. Reconstrói o stock global e calcula as quantidades a repor em todas as regiões no mínimo ou abaixo dele.
5. Regista a ordem em todos os armazéns envolvidos de forma atómica, com Two-Phase Commit.
6. Envia a ordem ao fornecedor pela rota REST/JSON ou SOAP/XML, conforme o tipo do fornecedor.
7. Guarda a fatura PDF no HDFS e regista a auditoria com a hora sincronizada pelo algoritmo de Berkeley.

Resultado obtido. Com Maputo e Beira a venderem o produto P2 (fornecedor F1) até ao mínimo, foi enviada uma única ordem de 80 unidades pela rota REST (40 para cada região), registada nos dois armazéns por 2PC, com a fatura no HDFS e replicação 3. Os pedidos seguintes dos dois nós entraram na secção crítica, encontraram a ordem já registada e não duplicaram a compra. Num segundo cenário, Nampula comprou o produto P4 ao fornecedor F2 pela rota SOAP. Todos os eventos ficaram na auditoria com a hora sincronizada.

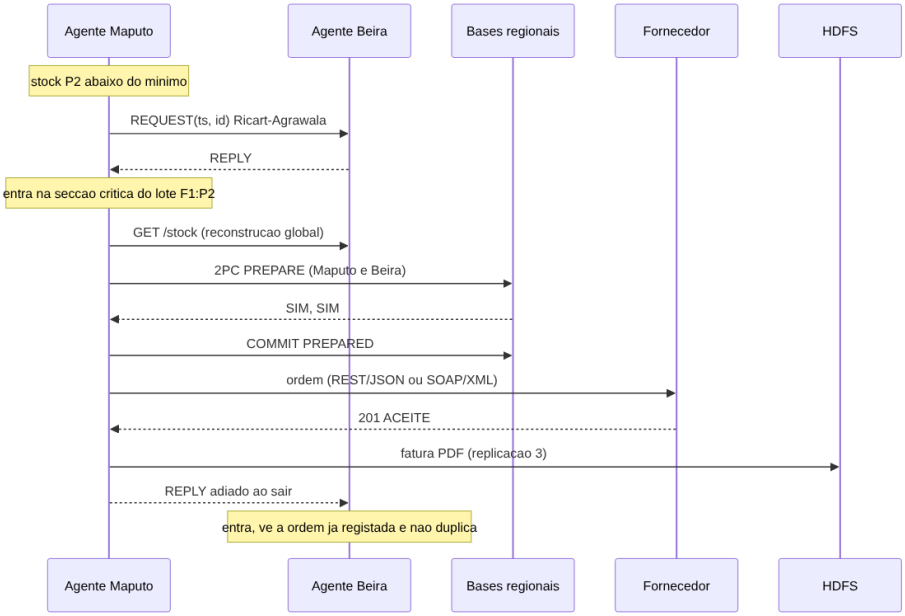


Figura 2 — Sequência de uma compra automática com dois nós em concorrência

2.2 Utilização do sistema

O sistema é operado numa interface web, disponível em qualquer um dos três nós (portas 8081, 8082 e 8083), com duas áreas. A **área do cliente** destina-se a pessoas particulares e a instituições (hospitais, bancos, escolas): o cliente cria conta, escolhe o armazém, junta um ou vários produtos e paga (M-Pesa, e-Mola, cartão ou transferência), recebendo um recibo com referência. A **área de gestão** permite registar produtos com o preço e, para cada armazém, o stock inicial e o stock mínimo (alterável depois), registar vendas e entradas, ver os clientes e o que compraram, acompanhar as compras ao fornecedor, abrir as faturas, os relatórios e os logs guardados no HDFS e ver o estado dos armazéns e serviços.

O sistema arranca vazio: só existem os armazéns e os fornecedores, e os produtos são registados na gestão. Um pedido baixa o stock do armazém escolhido numa só transação local (todos os produtos ou nenhum), com o preço lido da base de dados. Ao atingir o mínimo, o agente compra sozinho; a ordem passa a ENVIADA quando o fornecedor a aceita, ou a FALHOU, sem fatura, se ele não responder, repetindo-se a compra de 30 em 30 s. A mercadoria chega 30 s depois: o stock sobe e a ordem passa a RECEBIDA. O pagamento é simulado, para demonstração: não liga a nenhum operador nem pede dados de cartão. As figuras 3 e 4 mostram as duas áreas.



Figura 3 — Área de gestão: receita dos pedidos, indicadores e estado do sistema

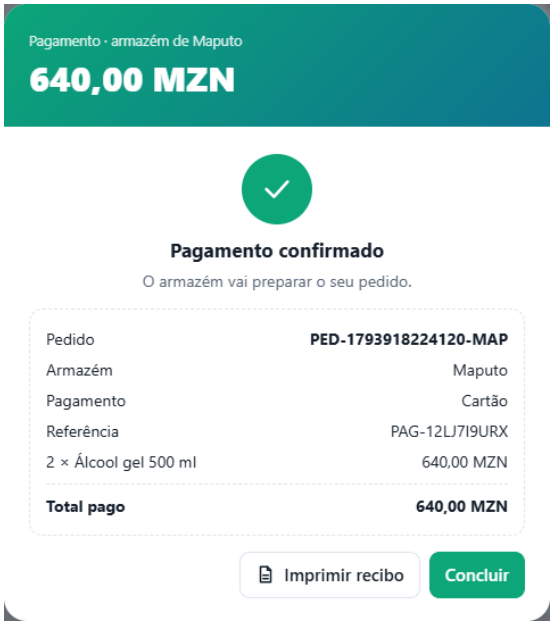


Figura 4 — Área do cliente: pagamento confirmado e recibo

3. Sistemas Operativos Distribuídos

3.1 Exclusão mútua: algoritmo de Ricart-Agrawala

Decisão. Entre os algoritmos sugeridos (Ricart-Agrawala e Anel), foi escolhido o **Ricart-Agrawala** [3] por três razões: não depende de um testemunho que se possa perder, atende os pedidos pela ordem em que foram feitos (justiça) e o seu custo, 2(N−1) mensagens por entrada, é baixo para três nós (4 mensagens).

Implementação. Cada recurso tem uma secção crítica própria, identificada pelo par `fornecedor:produto` (por exemplo, `F1:P2`), para que compras de lotes diferentes não se bloqueiem. Cada nó mantém um relógio lógico de Lamport [4] e um estado por recurso (LIVRE, A_PEDIR ou DENTRO):

- para entrar, o nó incrementa o relógio e envia `REQUEST(ts, id)` aos restantes nós;
- ao receber um pedido, responde de imediato se estiver LIVRE ou se o pedido recebido for mais antigo; caso contrário adia o REPLY. Os empates de timestamp são resolvidos pelo identificador do nó (Maputo = 1, Beira = 2, Nampula = 3);

- entra quando recebe REPLY de todos os nós vivos e, ao sair, envia os REPLY adiados.

Tolerância a falhas. O algoritmo original bloqueia se um nó falhar, porque nunca recebe o seu REPLY. Para o evitar, usa-se o detetor de falhas por heartbeat (pedido `/estado` a cada 5 s com timeout de 2 s): os nós dados como em falha são retirados da espera, e existe ainda um timeout global de 20 s.

Resultado obtido. Com Maputo e Beira a pedir o lote `F1:P2` em simultâneo, ambos com ts = 1, o empate foi resolvido pelo identificador: Maputo entrou primeiro e adiou o REPLY à Beira; a Beira só entrou quando Maputo saiu, cerca de 5 s depois. Com o agente de Nampula desligado, a exclusão mútua continuou a funcionar e Nampula foi ignorado por estar em falha.

3.2 Sincronização de relógios: algoritmo de Berkeley

Decisão. Todos os contentores partilham o relógio do mesmo kernel; com NTP (Network Time Protocol) não haveria desvio para corrigir nem resultado observável. Optou-se pelo **algoritmo de Berkeley** [5], implementado em Java, em que cada nó tem um desvio simulado (Maputo 0 s, Beira +25 s, Nampula −10 s).

Funcionamento. O agente de Maputo é o mestre (time daemon). Em cada ronda: pede a hora a cada nó; compensa metade do tempo de ida e volta (RTT); calcula a diferença de cada relógio; ignora diferenças superiores a 10 minutos; calcula a média e envia a cada nó o **ajuste relativo** (média − diferença), que se mantém correto independentemente do atraso da mensagem. As rondas são automáticas, de 30 em 30 s, e podem também ser pedidas na interface.

Nó	Desvio inicial	Diferença	Ajuste	Desvio final medido
Maputo (mestre)	0 ms	0 ms	+5 000 ms	5 000 ms
Beira	+25 000 ms	+25 000 ms	−20 000 ms	5 001 ms
Nampula	−10 000 ms	−10 000 ms	+15 000 ms	5 000 ms

Tabela 2 — Ronda de Berkeley: média das diferenças = +5 000 ms

Todas as transações e auditorias passam a ser registadas com a hora corrigida. A diferença residual de 1 ms deve-se ao tempo de rede. O mestre é fixo; a eleição de um novo mestre (por exemplo, pelo algoritmo Bully) fica como trabalho futuro.

4. Sistemas de Bases de Dados Distribuídos

4.1 Modelo de dados e alocação

A Tabela 3 apresenta o modelo da base de dados distribuída [6]: os dados de referência e os dados que têm de ser iguais em todos os nós estão replicados; os dados de cada armazém estão fragmentados por região.

Tabela	Alocação	Colunas e restrições
`armazem`, `fornecedor`	Replicadas nos 3 nós	armazem(id PK, nome, regiao); fornecedor(id PK, nome, tipo_integracao REST|SOAP, endpoint)

Tabela	Alocação	Colunas e restrições
produto	Replicada nos 3 nós (escrita por 2PC)	id PK; nome; unidade; preco ≥ 0 (MZN por unidade)
cliente	Replicada nos 3 nós (escrita por 2PC)	id PK; nome; tipo (particular, hospital, banco...); email UNIQUE; senha_hash (PBKDF2)
stock	Fragmentada por região	PK (produto_id, regiao); quantidade ≥ 0; limite_minimo; FK produto e fornecedor; CHECK regiao = região do nó
ordem_compra	Fragmentada por região	id PK; FK produto e fornecedor; regiao (CHECK); quantidade > 0; estado CONFIRMADA, ENVIADA, RECEBIDA ou FALHOU; criada_em
pedido_cliente	Fragmentada por região	id PK; pedido; FK cliente e produto; regiao (CHECK); quantidade > 0; preco_unitario; criado_em
pagamento	Fragmentada por região	pedido PK; FK cliente; regiao (CHECK); metodo; referencia; valor ≥ 0; estado
auditoria	Local a cada nó	id PK; no_origem; hora (Berkeley); operacao; detalhe. Exportada para o HDFS a cada minuto

Tabela 3 — Modelo da base de dados distribuída

4.2 Fragmentação horizontal

A tabela de stock é fragmentada por região: STOCK_MAPUTO = σ(regiao = 'Maputo')(STOCK), e o mesmo para a Beira e Nampula; as ordens, os pedidos dos clientes e os pagamentos seguem a mesma regra. A tabela global obtém-se pela união dos três fragmentos. As três regras de correção foram verificadas:

- **Disjunção**, garantida pela própria base de dados com `CHECK (regiao = 'Maputo')`: a inserção de uma linha da Beira no fragmento de Maputo foi recusada com o erro SQLSTATE 23514 (`check_violation`);
- **Reconstrução e completude**: com os quatro produtos de demonstração, o endpoint `/stock/global` devolveu as 12 linhas esperadas (4 produtos × 3 regiões);
- **Falha parcial**: com o agente da Beira desligado, a consulta global devolveu 8 linhas e indicou o fragmento indisponível, mantendo o sistema disponível.

4.3 Replicação mestre-escravo

Cada região tem um servidor mestre e uma réplica. A réplica é criada com `pg_basebackup` e recebe continuamente o WAL (Write-Ahead Log) do mestre por streaming, ficando disponível só para leitura (hot standby) [12]. A replicação é assíncrona: tem menor latência, com o risco de perder as últimas escritas se o mestre falhar.

O failover é feito no cliente: o endereço JDBC lista os dois servidores com `targetServerType=preferPrimary`. Nos testes, a réplica mostrou o estado streaming em `pg_stat_replication`; uma alteração no mestre foi visível na réplica; a réplica recusou escritas; com o mestre da Beira parado, o agente passou automaticamente para `db-beira-rep` em modo só leitura e, após `pg_promote()`, voltou a aceitar escritas. Quando o mestre volta sem

ter havido promoção, o nó regressa a ele em cerca de 1 minuto, porque as ligações à base de dados são renovadas a cada minuto. Foi também observado o problema de **split-brain** quando o antigo mestre volta depois de uma promoção sem ser reconstruído como réplica, o que justifica a regra operacional de não o reativar diretamente.

4.4 Consistência: Two-Phase Commit

Decisão. O enunciado admite 2PC ou Paxos/Raft. O 2PC [7] é um protocolo de compromisso atómico que exige unanimidade, o que corresponde ao requisito: uma compra multi-armazém fica registada em todos os armazéns envolvidos ou em nenhum. Paxos e Raft são protocolos de consenso por maioria, adequados à replicação de um log, não à atomicidade entre fragmentos distintos.

Implementação. O agente que inicia a operação é o coordenador; os agentes das regiões envolvidas são participantes. Na fase 1, cada participante executa a sua parte e `PREPARE TRANSACTION`, que grava a transação em disco (voto SIM); qualquer erro provoca `ROLLBACK` (voto NÃO), e a ausência de resposta conta como NÃO. O coordenador grava a decisão na auditoria **antes** da fase 2 e envia `COMMIT PREPARED` ou `ROLLBACK PREPARED`. O mesmo mecanismo é usado nas compras multi-armazém, no registo, edição e remoção de produtos e no registo de contas de clientes, que têm de ficar iguais nos três nós. O coordenador envia ainda o ABORT a quem não respondeu, porque a preparação pode ter terminado depois do timeout. Como salvaguarda, um participante com uma transação preparada há mais de 120 s pergunta a decisão aos outros nós e conclui-a; se o coordenador iniciou a transação há mais de 5 minutos e nunca decidiu, aplica-se o aborto presumido; sem decisão conhecida (coordenador em baixo), a transação espera, porque o 2PC não pode adivinhar.

Teste	Situação	Resultado
1	Maputo e Beira votam SIM	COMMIT; a mesma ordem existe nas duas bases
2	Beira vota NÃO (quantidade 0 viola CHECK)	ABORT; Maputo desfaz a sua parte
3	Agente da Beira desligado	NÃO por ausência de resposta; ABORT
4	Coordenador falha após a decisão	Transações presas em `pg_prepared_xacts` (bloqueio); a decisão COMMIT é lida do log e concluída nas duas bases, manualmente ou, ao fim de 120 s, pela terminação automática
5	Transação preparada cuja decisão foi ABORT	A terminação automática lê ABORT e executa ROLLBACK PREPARED
6	Registar um produto com o agente da Beira desligado	ABORT; o produto não fica em nenhum armazém e não restam transações preparadas

Tabela 4 — Testes do 2PC

5. Sistema de Ficheiros Distribuído

Decisão. Entre as opções do enunciado, o NFS (Network File System) tem um único servidor, logo não tolera a sua queda; o GlusterFS exige módulos de kernel e contentores privilegiados,

pouco viáveis em Docker Desktop. Foi escolhido o **HDFS** (Hadoop Distributed File System) [8] com 1 NameNode e 3 DataNodes e **fator de replicação 3**.

O NameNode guarda os metadados e os DataNodes os blocos; cada bloco é copiado em pipeline para os três DataNodes. Os agentes acedem por WebHDFS: o NameNode indica o DataNode e os dados seguem diretamente para ele. São guardados no DFS as faturas PDF (`/scad/faturas`), os logs de erros de cada nó (`/scad/logs/erros-<região>.log`) e os relatórios diários de auditoria em CSV (`/scad/auditoria`), atualizados a cada minuto. Cada nó escreve os seus próprios ficheiros, porque o HDFS só admite um escritor de cada vez por ficheiro. Se o HDFS estiver indisponível, as faturas e os erros ficam em fila e são gravados quando ele voltar, sem atrasar a compra.

Resultados. O `hdfs fsck` mostrou a fatura com `Live_repl=3` e estado HEALTHY; a fatura escrita por Maputo foi lida pela Beira e listada por Nampula (transparência de acesso e de localização); com o agente de Maputo e o `datanode1` desligados, a Beira continuou a ler a fatura; dois nós acrescentaram linhas ao mesmo ficheiro de log. Com um DataNode parado imediatamente antes de uma compra, a fatura foi guardada nas restantes cópias; com o NameNode parado, a fatura ficou em fila e foi gravada automaticamente quando ele voltou. O NameNode é um ponto único de falha (os dados não se perdem, mas ficam inacessíveis); a configuração de alta disponibilidade fica fora do âmbito.

6. Java / CORBA

A comunicação interna entre o módulo central de auditoria e os robôs de stock regionais usa **JacORB 3.9** [9]. O JacORB foi executado sobre Java 8, a plataforma para a qual esta versão foi desenvolvida; o resto do sistema usa Java 21. A interface IDL (Interface Definition Language), no ficheiro `corba/idl/scad.idl`, é a seguinte:

```
module SCAD {
    // Alerta enviado por um robo quando um produto atinge o stock minimo
    struct AlertaStock {
        string    produtoId;
        string    regiao;
        long      quantidadeAtual;
        long      limiteMinimo;
        long long timestamp;          // milissegundos desde 1970 (UTC)
    };
    typedef sequence<AlertaStock> ListaAlertas;
    exception ProdutoInexistente { string motivo; };

    // Implementado por cada robo de stock regional; invocado pela auditoria
    interface AgenteStock {
        readonly attribute string regiao;
        long consultarSaldo(in string produtoId) raises (ProdutoInexistente);
    };

    // Implementado pelo modulo central de auditoria; invocado pelos robos
    interface Auditoria {
        oneway void dispararAlerta(in AlertaStock alerta); // o robo nao bloqueia
        ListaAlertas listarAlertas();
    };
};
```

O compilador IDL do JacORB gera, a partir deste ficheiro, os stubs (cliente), os skeletons POA (Portable Object Adapter, servidor), os helpers e os holders. A arquitetura é:

- `corba-naming`: Naming Service do JacORB, cuja referência (IOR — Interoperable Object Reference) é partilhada por um volume;
- `auditoria`: servant de Auditoria, registado como "Auditoria";
- `robo-*`: cada robô é servidor de AgenteStock (registado como "AgenteStock-<Região>") e cliente da auditoria; de 10 em 10 s verifica o stock e invoca `dispararAlerta`, declarada `oneway` para que o robô não bloqueie à espera da auditoria.

Resultados. Os quatro objetos registaram-se no Naming Service; cada robô dispara um alerta quando um produto atinge o stock mínimo; ao baixar o stock de P1 em Nampula, o alerta chegou à auditoria em menos de 10 s; `consultarSaldo` respondeu em 40 a 100 ms por chamada IIOP (Internet Inter-ORB Protocol); com o robô da Beira desligado, a chamada falhou com a exceção `TRANSIENT` e os restantes robôs continuaram a responder.

7. Serviços Web Distribuídos

O envio final da ordem ao fornecedor tem duas rotas, escolhidas pelo campo `tipo_integracao` da tabela `fornecedor` (Tabela 5) [10, 11]:

Aspeto	**REST/JSON (fornecedor moderno)**	**SOAP/XML (fornecedor legado)**
Endpoint	`POST /api/ordens`	`POST /ws` (envelope SOAP 1.1)
Contrato	Recursos e códigos HTTP	WSDL em `/ws?wsdl` (document/literal, XSD)
Sucesso	201 Created (nova); 200 OK (repetida)	`EnviarOrdemResponse` com estado ACEITE
Erro de pedido	400 Bad Request	`soap:Fault` com faultcode `soap:Client`

Tabela 5 — Rotas de integração com fornecedores

Robustez. O cliente usa timeouts (2 s de ligação, 3 s de leitura), faz até 3 tentativas com espera crescente (1 s, 2 s) em falhas de rede ou erros 5xx, não repete pedidos inválidos (4xx ou `soap:Client`) e garante **idempotência** pelo identificador da ordem: o fornecedor que já conhece o `ordemId` devolve a resposta original em vez de criar outra ordem. Os documentos XML são lidos com DTD desativado, para prevenir ataques XXE (XML External Entity).

Resultados. O WSDL foi publicado; a mesma ordem REST devolveu 201 e depois 200; a quantidade 0 devolveu 400 em REST e SOAP Fault em SOAP, sem novas tentativas; com o fornecedor REST desligado, houve 3 tentativas falhadas e, após a recuperação, o reenvio da mesma ordem devolveu 201 e uma segunda repetição devolveu 200, confirmando que a ordem não foi duplicada.

8. Testes de tolerância a falhas

A Tabela 6 resume os testes de resiliência, feitos com o sistema em funcionamento.

Falha simulada	Comando	Comportamento observado
Agente regional	`docker compose stop agente-beira`	O heartbeat deteta a falha em menos de 5 s; Ricart-Agrawala ignora o nó; a região deixa de vender; registar um produto é recusado em todos os armazéns (2PC aborta)
Mestre de uma região	`docker compose stop db-beira`	Leituras continuam na réplica e as vendas são recusadas; ao voltar o mestre, o nó regressa em cerca de 1 minuto; em alternativa, `pg_promote()` repõe as escritas
Coordenador do 2PC	Falha simulada entre fases	Participantes bloqueados; recuperação pela decisão gravada, manual ou automática ao fim de 120 s
DataNode e agente escritor	`docker compose stop agente-maputo datanode1`	A fatura continua legível a partir da Beira
NameNode	`docker compose stop namenode`	A fatura fica em fila e é gravada quando o NameNode volta
Robô de stock	`docker compose stop robo-beira`	`TRANSIENT` para esse robô; os restantes respondem
Fornecedor	`docker compose stop fornecedor-rest`	3 tentativas; ordem FALHOU, sem fatura; erro no HDFS; a compra repete-se sozinha de 30 em 30 s, sem duplicar

Tabela 6 — Resumo dos testes de resiliência

Os testes podem ser repetidos com os scripts da pasta `testes/` do repositório, depois de carregar os produtos de demonstração (`zerar.ps1 -Demo`).

9. Limitações e trabalho futuro

- Todos os nós correm numa só máquina física: é simulada a falha de processos, não a falha do anfitrião. A configuração permite mover um nó para outro computador alterando apenas endereços.
- O coordenador do 2PC e o NameNode do HDFS continuam a ser pontos únicos de falha; o 3PC, um protocolo de consenso ou o HDFS em alta disponibilidade seriam a evolução natural.
- O mestre de Berkeley é fixo e o failover da base de dados exige a promoção manual da réplica. Até lá, essa região não regista vendas: o sistema dá prioridade à consistência sobre a disponibilidade dessa região, enquanto as outras continuam a funcionar.
- Com um armazém em baixo não é possível criar contas nem alterar produtos (2PC), mas as contas existentes continuam a entrar.
- O pagamento é simulado e a entrega da mercadoria é simulada pelo SCAD, 30 s depois de o fornecedor aceitar; as ordens recebidas pelos fornecedores simulados são mantidas em memória.
- Com o HDFS em baixo, faturas e erros ficam em fila na memória do agente: se esse agente reiniciar antes de o HDFS voltar, o que estava em fila perde-se.

- As rotas entre nós não exigem autenticação: o sistema assume uma rede interna. O login protege as áreas de cliente e de gestão.

10. Conclusão

O SCAD cumpre as cinco componentes do enunciado com mecanismos efetivamente implementados e testados: exclusão mútua por Ricart-Agrawala e sincronização por Berkeley; fragmentação horizontal em três nós com replicação mestre-escravo e atomicidade por 2PC; armazenamento de faturas, relatórios e logs em HDFS com replicação 3; comunicação interna por CORBA/JacORB com uma IDL própria; e integração com fornecedores por REST/JSON e SOAP/XML. Os testes mostraram que a falha de um nó (agente, base de dados, DataNode, NameNode, robô ou fornecedor) não compromete a consistência dos dados nem a disponibilidade do resto do sistema, e evidenciaram também as limitações clássicas destes protocolos, como o bloqueio do 2PC e o split-brain na replicação.

O sistema é utilizável de ponta a ponta através de uma interface web: os clientes fazem e pagam pedidos aos armazéns, e a gestão acompanha o stock, os clientes, a receita e as compras automáticas que o SCAD faz aos fornecedores quando o stock atinge o mínimo.

11. Referências bibliográficas

[1] Tanenbaum, A. S., & van Steen, M. (2017). *Distributed systems* (3.ª ed.). distributed-systems.net.

[2] Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed systems: Concepts and design* (5.ª ed.). Addison-Wesley.

[3] Ricart, G., & Agrawala, A. K. (1981). An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM, 24*(1), 9–17.

[4] Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM, 21*(7), 558–565.

[5] Gusella, R., & Zatti, S. (1989). The accuracy of the clock synchronization achieved by TEMPO in Berkeley UNIX 4.3BSD. *IEEE Transactions on Software Engineering, 15*(7), 847–853.

[6] Özsu, M. T., & Valduriez, P. (2020). *Principles of distributed database systems* (4.ª ed.). Springer.

[7] Gray, J. (1978). Notes on data base operating systems. In *Operating Systems: An Advanced Course* (LNCS 60, pp. 393–481). Springer.

[8] Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop distributed file system. *IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 1–10.

[9] Object Management Group. (2012). *Common Object Request Broker Architecture (CORBA) specification, version 3.3*.

[10] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tese de doutoramento, University of California, Irvine].

[11] W3C. (2000). *Simple Object Access Protocol (SOAP) 1.1* (W3C Note).

[12] PostgreSQL Global Development Group. (2023). *PostgreSQL 16 documentation: High availability, load balancing, and replication*.

Anexo A — Instalação e utilização

Requisitos: Docker Desktop com pelo menos 8 GB de memória. Instalação e scripts principais (PowerShell, na pasta do projeto):

```
git clone https://github.com/Bill-Ars/scad.git
cd scad
docker compose up -d --build

# Recomeçar vazio; com produtos de exemplo; só um armazém
powershell -ExecutionPolicy Bypass -File .\zerar.ps1
powershell -ExecutionPolicy Bypass -File .\zerar.ps1 -Demo
powershell -ExecutionPolicy Bypass -File .\zerar-armazem.ps1 -Regiao Beira

# Exemplo de teste (depois de zerar.ps1 -Demo)
powershell -ExecutionPolicy Bypass -File .\testes\teste-integracao.ps1
```

Endereços: sistema (clientes e gestão) em http://localhost:8081, 8082 ou 8083, com a senha de gestão `admin1234`; ordens recebidas pelo fornecedor REST em http://localhost:8091/api/ordens; contrato SOAP em http://localhost:8092/ws?wsdl; interface do HDFS em http://localhost:9870.